

ASSIGNMET 1 : PROGRAM CODE

#Assignment 1. Perform the following operations using Python on the Facebook metrics data sets

a. Create data subsets

b. Merge Data

c. Sort Data

d. Transposing Data

e. Shape and reshape Data

#

Download Dataset (Facebook metrics) : from following link

<https://www.kaggle.com/datasets/masoodanzar/facebook-metrics>

```
import pandas as pd
```

```
import numpy as np
```

```
df = pd.read_csv('/home/mangal/Downloads/dataset_Facebook.csv', sep = ";")
```

```
df.head()
```

a. Create data subsets

```
subset1 = df.loc[df["Type"] == "Photo"]
```

```
subset1.to_csv('/home/mangal/Downloads/photo_data.csv')
```

#create subsets by selecting specific rows based on position

```
temp = df.iloc[10:25]
```

```
print(temp)
```

create subsets by selecting specific columns and specific rows

```
df1 = df[['Type', 'Category', 'comment']].loc[4:17]
```

```
df2 = df[['Type', 'Category', 'Paid']].loc[24:30]
```

```
df3 = df[['Type', 'Category', 'Paid']].loc[31:35]
```

```
print("\n", "DF 1 ", df1, "\n")
```

```
print("\n", "DF 2 ", df2, "\n")
```

```
print("\n", "DF 3 ", df3, "\n")
```

b. Merge Data

Merge refers to combining subsets of data

```
pd.concat([df1, df2, df3])
```

c. Sort Data

```
df.sort_values(['Category', 'Paid'], ascending=False)
```

d. Transposing Data

```
df.transpose() # or simply df.T
```

e. Shape and reshape Data

```
df_melted = df1.melt(id_vars = None , value_vars = None , ignore_index = False )
```

```
print(df_melted)
```

pivot

```
df_pivoted = df_melted.pivot( columns = 'variable', values = 'value')
```

```
print( df_pivoted)
```

ASSIGNMET 2 : PROGRAM CODE (part a, b c, d)

#Assignment 2. Perform the following operations using Python on the Air quality and Heart
#Diseases data sets

a. Data cleaning

b. Data integration

c. Data transformation

d. Error correcting

e. Data model building

Dataset (Air quality): <https://www.kaggle.com/datasets/fedesoriano/air-quality-data-set>

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
df = pd.read_csv('/home/mangal/Downloads/AirQuality.csv',sep = ';')
```

```
# check the columns names and the respective datatypes
df.info()
```

```
# check the statistical informatin of all columns: std , mean , min , max
df.describe()
```

```
# Examine some sample rows ; head shows top five rows by default
df.head()
```

```
# Remove , from the columns 'CO(GT)' , 'C6H6(GT)' , 'T' , 'RH' and 'AH' all
```

```
# handling missing values, And dropping duplicate rows
```

```
df.isnull().sum()
```

```
# The above output shows that the last two columns contain entirely NaN
```

```
df['Unnamed: 15'].isnull().sum()
```

```
df['Unnamed: 16'].isnull().sum()
```

```
df.drop(['Unnamed: 15','Unnamed: 16'],axis = 1,inplace = True)
```

```
# We will print all the rows having any of its value as NaN
```

```
df[df.isnull().any(axis=1)]
```

```
df.dropna(how = 'all', inplace = True)
```

```
column_list = df.columns.values.tolist()
```

```
print(column_list)
```

```
# Lets check unique values in each columns
```

```
# column_list = df.columns.values.tolist()
```

```
for column_name in column_list:
    print ("\n", column_name)
    print ( df[column_name].value_counts(dropna = False ) ) # print count ALL unique values in
each column
```

```
## ERROR CORRECTION
```

```
#remove , from the column 'CO(GT)', 'C6H6(GT)', 'T' , 'RH' , 'AH'
```

```
df.info()
print(df['CO(GT)'])
print( df['C6H6(GT)'])
print(df['RH'])
print(df['AH'])
print(df['T'])
```

```
j = 'CO(GT) C6H6(GT) T RH AH'.split()
print(j)
df.replace(to_replace=',',value='.',regex=True,inplace=True)
for i in j :
    df[i] = pd.to_numeric(df[i],errors='coerce')
```

```
## DATA TRANSFORMATION
```

```
df.head()
scaler = StandardScaler()
```

```
Numerical_col = df.select_dtypes(exclude = [np.object_ , np.datetime64 ] ) # or use include
np.float64
```

```
#categorical_col = dataframe.select_dtypes(exclude=np.number)
```

```
for col in Numerical_col:
    df[[col]] = scaler.fit_transform(df[[col]])
df.head()
```

```
df.info()
```

```
# DATA TRANSFORMATION :
# changing the form of the data
```

```
df['Date'] = pd.to_datetime(df['Date'],dayfirst=True)
df['Time'] = pd.to_datetime(df['Time'],format= '%H.%M.%S' ).dt.time
```

ASSIGNMET 2 : PROGRAM CODE (part e)

#Assignment 2. Perform the following operations using Python on the Air quality and Heart

#Diseases data sets

a. Data cleaning

b. Data integration

c. Data transformation

d. Error correcting

e. Data model building

Dataset (Heart Diseases) : <https://www.kaggle.com/datasets/johnsmith88/heart-disease-dataset>

```
import numpy as np
```

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import accuracy_score
```

```
# loading the csv data to a Pandas DataFrame
```

```
heart_data = pd.read_csv('/home/mangal/Downloads/heart.csv')
```

```
heart_data.head()
```

```
heart_data.tail()
```

```
heart_data.shape
```

```
heart_data.info()
```

```
heart_data.isnull().sum()
```

```
heart_data.describe()
```

```
heart_data['target'].value_counts()
```

```
# 1 --> Defective Heart
```

```
#
```

```
# 0 --> Healthy Heart
```

```
#
```

```
# Splitting the Features and Target
```

```
X= heart_data.drop(columns='target',axis=1)
```

```
Y=heart_data['target']
```

```
print(X)
```

```
print(Y)
```

```
# Splitting the Data into Training Data and Test Data
```

```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=2)
```

```

print(X.shape, X_train.shape, X_test.shape)

# Model Training
# Logistic Regression

model = LogisticRegression(max_iter=1050)

# training the LogisticRegression model with Training data
model.fit(X_train, Y_train)

# Model Evaluation: finding accuracy on training data
X_train_prediction = model.predict(X_train)

training_data_accuracy = accuracy_score(X_train_prediction, Y_train)
print('Accuracy on training data : ', training_data_accuracy)

# accuracy on test data
X_test_prediction = model.predict(X_test)
test_data_accuracy = accuracy_score(X_test_prediction, Y_test)
print('Accuracy on Test data : ', test_data_accuracy)
type(X_test)
print(X_test_prediction)

# Building a Predictive System
input_data = (52,1,0,125,212,0,1,168,0,1,2,2,3) # single instance of 13 features

# change the input data to a numpy array
input_data_as_numpy_array= np.array(input_data)

# reshape the numpy array as we are predicting for only on instance
# so we want to reshape it to 1*13 : 1 sample 13 features
input_data_reshaped = input_data_as_numpy_array.reshape(1,-1)

test_df = pd.DataFrame(input_data_reshaped, columns = X_test.columns )
prediction = model.predict(test_df)
print(prediction)

if (prediction[0]== 0):
    print('The Person does not have a Heart Disease')
else:
    print('The Person has Heart Disease')

```

ASSIGNMENT 3 : PROGRAM CODE

Visualize the data using Python libraries matplotlib, seaborn by plotting the graphs for Air quality
#and Heart Diseases datasets

Dataset(Heart Disease) : <https://www.kaggle.com/datasets/johnsmith88/heart-disease-dataset>

```
import pandas as pd
import numpy as np
df=pd.read_csv("/home/mangal/Downloads/heart.csv") # use your path
```

```
df.shape
df.columns
df.dtypes
df.info()
df.describe()
df.isnull().sum()
# no null values
df.duplicated().any().sum()
df.drop_duplicates(inplace=True)
df.shape
df.duplicated().sum() #chec duplicates have been removed
```

```
# Heatmaps in Seaborn can be plotted by using the seaborn.heatmap()method.
import matplotlib.pyplot as plt
import seaborn as sns
```

```
plt.figure(figsize=(15,6)) # try (15 , 10)
sns.heatmap(df.corr(),annot=True)
#sns.heatmap(df.corr(),annot=True,linewidths=0.5,fmt= ".2f",cmap="YlGnBu");
plt.title('Degree of Correlation of variables in the dataset')
```

here , we can see that the 'cp','thalach','slope' columns show a strong positive correlation with the
#target variable which means that the type of chest plain (cp)

```
# ##### Discrete variable : use seaborn.countplot() or histplot()
# ##### Distribution of target variable using seaborn.countplot()
# Countplot
sns.countplot(x='target',data=df)
plt.xticks([0,1],['less chance','more chance'])
plt.title('Chances of heart disease')
plt.figure(figsize=(15,6))
df['target'].value_counts(normalize=True)
sns.countplot(x='sex',data=df )
plt.title('Number of males and females')
plt.xticks([0,1] , ['females','males'])
plt.show()
```

```
sns.countplot(x='sex',data=df, hue ='target' )
plt.title('chances of heart disease by gender')
plt.xticks([0,1],['females','males'])
plt.legend(labels=['less chance','high chance'])
```

```
sns.histplot(df['cp'])
plt.xticks([0,1,2,3],["typical angina","atypical angina","non-anginal pain","asymptomatic"])
plt.xticks(rotation=70)
plt.figure(figsize=(10,7))
plt.show()
```

```
list1 = list(df['cp'].value_counts(normalize=True))
list1
```

```
plt.pie(list1,labels=["typical angina","non-anginal", "atypical angina", "asymptomatic"],
startangle=180, shadow=True, autopct='%1.1f%%')
plt.show()
```

`##### Distribution of the chest pain distribution as per the target variable`

```
sns.countplot(x='cp',hue='target',data=df)
# formatting the plot
plt.title('Relation between types of chest pain and number of people having high or low chances of heart attack')
plt.xticks([0,1,2,3],["typical angina","atypical angina","non-anginal pain","asymptomatic"])
plt.legend(labels=['low chance','high chance'])
```

```
sns.countplot(x='fbs' , hue='target',data=df)
plt.legend(labels=['low chance','high chance'])
# There are a large number of people who are not diabetic but have high chances of heart attack as
# compared to people who are diabetic , in this dataset.
```

```
df['fbs'].value_counts(normalize=True)
# around 85% of the people are non- diabetic and only 14 % of the people are diabetic.
```

```
## ***Visualisation of resting blood pressure in males and females***
g= sns.FacetGrid(df,hue="sex",aspect=4)
g.map(sns.kdeplot,'restbps',shade=True)
plt.xlabel("resting blood pressure")
plt.legend(labels=["female","male"])
```

```
## ***Visualisation of serum cholesterol in males and females***
chol= sns.FacetGrid(df,hue="sex",aspect=4)
chol.map(sns.kdeplot,'chol',shade=True)
plt.xlabel("serum cholesterol")
plt.legend(labels=["female","male"])
```

ASSIGNMENT 4 : Program Code

#HADOOP INSTALLATION

(1.)

sudo apt update

2.)

sudo apt install openjdk-8-jdk

3.)

sudo nano .bashrc

Note : In this editor you can only use arrows keys

copy the text in following block BLOCK_1 and paste it at the end of above file

use command "shift + control + v" to copy paste ;

save file using "control o" () ;# exit using control x

#BLOCK_1 ++++++

export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64

export PATH=\$PATH:/usr/lib/jvm/java-8-openjdk-amd64/bin

export HADOOP_HOME=~/.hadoop-3.2.3/

export PATH=\$PATH:\$HADOOP_HOME/bin

export PATH=\$PATH:\$HADOOP_HOME/sbin

export HADOOP_MAPRED_HOME=\$HADOOP_HOME

export YARN_HOME=\$HADOOP_HOME

export HADOOP_CONF_DIR=\$HADOOP_HOME/etc/hadoop

export HADOOP_COMMON_LIB_NATIVE_DIR=\$HADOOP_HOME/lib/native

export HADOOP_OPTS="-Djava.library.path=\$HADOOP_HOME/lib/native"

export HADOOP_STREAMING=\$HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.2.3.jar

export HADOOP_LOG_DIR=\$HADOOP_HOME/logs

export PDSH_RCMD_TYPE=ssh

+++++

4.)

sudo apt-get install ssh

5.) # Download hadoop-3.2.3 as follows# search "apache hadoop download" on google : follow the topmost link pointing to apache website

Here on this page somewhere you will find a link to archives : click on the archives link

On archives page It provides you a dircetory of hadoop installers of various version

choose directory "hadoop-3.2.3"

Here download the following file : hadoop-3.2.3.tar.gz

let the download finish ; then extract the file using following command

tar -zxvf ~/Downloads/hadoop-3.2.3.tar.gz

Note: instead of tilde sign in above , you may give full path of the downloaded installer

6.) # change the directory as follows

cd hadoop-3.2.3/etc/hadoop

7) # open hadoop-env.sh using following command

sudo nano hadoop-env.sh

here in this file you will find the line like JAVA_HOME=/usr/java/testing hdfsd fs -ls

go to this line and remove the comment of this line also modify this line as shown below

note after removing comment the line should not have any leading spaces

JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64# Additionally one more line in above file has

JAVA_Home: Do the similar modifications there

save this file after above two modifications and exit

8.) # open file core-site.xml as follows

sudo nano core-site.xml

copy the text in the following block BLOCK2 and paste it in above file in between

```
<configuration>
```

```
# tags
```

```
#BLOCK2 #####
```

```
<configuration>
```

```
<property>
```

```
<name>fs.defaultFS</name>
```

```
<value>hdfs://localhost:9000</value> </property>
```

```
<property>
```

```
<name>hadoop.proxyuser.dataflair.groups</name> <value>*</value>
```

```
</property>
```

```
<property>
```

```
<name>hadoop.proxyuser.dataflair.hosts</name> <value>*</value>
```

```
</property>
```

```
<property>
```

```
<name>hadoop.proxyuser.server.hosts</name> <value>*</value>
```

```
</property>
```

```
<property><name>hadoop.proxyuser.server.groups</name> <value>*</value>
```

```
</property>
```

```
</configuration>
```

```
#####
```

9.) # open file hdfs-site.xml as follows

sudo nano hdfs-site.xml

copy the text in the following block BLOCK3 and paste it in above file in between

```
<configuration>
```

```
# tags
```

```
# BLOCK3*****
```

```
<configuration>
```

```
<property>
```

```
<name>dfs.replication</name>
```

```
<value>1</value>
```

```
</property>
```

```
</configuration>
```

```
*****
```

10.) # open the file mapred-site.xml as follows

sudo nano mapred-site.xml

copy the text in the following block BLOCK4 and paste it in above file in between

```
<configuration>
```

```
# tags# BLOCK4 %%%%%%%%%%  
%
```

```
<property>
```

```

<name>mapreduce.framework.name</name> <value>yarn</value>
</property>
<property>
<name>mapreduce.application.classpath</name>
<value>$HADOOP_MAPRED_HOME/share/hadoop/mapreduce/*:
$HADOOP_MAPRED_HOME/
share/hadoop/mapreduce/lib/*</value>
</property>
%%%%%%%%%%%%%%
11.) # open the file yarn-site.xml as follows
sudo nano yarn-site.xml
# copy the text in the following block BLOCK5 and paste it in above file in between
<configuration>
# tags
# BLOCK 5=====
<property><name>yarn.nodemanager.aux-services</name>
<value>mapreduce_shuffle</value>
</property>
<property>
<name>yarn.nodemanager.env-whitelist</name>
<value>JAVA_HOME,HADOOP_COMMON_HOME,HADOOP_HDFS_HOME,HADOOP_CON
F_DIR,CLASSPATH_PREPEND_DISTCACHE,HADOOP_YARN_HOME,HADOOP_MAPRED
_HOME</value>
</property>
=====
12.)
# connecting to ssh ; here give your Systems (OS user) password when asked
ssh localhost
(13.)
sudo service ssh restart
14.)
ssh-keygen -t rsa -P "" -f ~/.ssh/id_rsa
15.)cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
16.)
chmod 0600 ~/.ssh/authorized_keys
17.)
hadoop-3.2.3/bin/hdfs namenode -format
18.)
#format the HDFS file system using following command #hdfs namenode -format
export PDSH_RCMD_TYPE=ssh
19.)
# start hadoop as follows
start-all.sh
20.)
# check your Hadoop installtion by entering following command
jps# It should show all the following dameons running

```

NameNode

DataNode

Secondary Name Node

Resource Manager

Node Manager

21.)

OR goto browser to check HDFS directory Access Hadoop web UI (open in browser)

<http://localhost:9870>

22.)

Or you may test your HDFS by creating directories and files by running follwing commands at

terminal

hadoop fs -mkdir /user

hadoop fs -mkdir /user/MNK

hadoop fs -put demo.csv /user/MNK

ASSIGNMENT 5 : Program Code

Map Reduce Program

SalesCountryDriver.java

```
package SalesCountry;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapred.*;

public class SalesCountryDriver {
    public static void main(String[] args) {
        JobClient my_client = new JobClient();
        // Create a configuration object for the job
        JobConf job_conf = new JobConf(SalesCountryDriver.class);

        // Set a name of the Job
        job_conf.setJobName("SalePerCountry");

        // Specify data type of output key and value
        job_conf.setOutputKeyClass(Text.class);
        job_conf.setOutputValueClass(IntWritable.class);

        // Specify names of Mapper and Reducer Class
        job_conf.setMapperClass(SalesCountry.SalesMapper.class);
        job_conf.setReducerClass(SalesCountry.SalesCountryReducer.class);

        // Specify formats of the data type of Input and output
        job_conf.setInputFormat(TextInputFormat.class);
        job_conf.setOutputFormat(TextOutputFormat.class);

        // Set input and output directories using command line arguments,
        //arg[0] = name of input directory on HDFS, and arg[1] = name of output directory to be
        //created to store the output file.

        FileInputFormat.setInputPaths(job_conf, new Path(args[0]));
        FileOutputFormat.setOutputPath(job_conf, new Path(args[1]));

        my_client.setConf(job_conf);
        try {
            // Run the job
            JobClient.runJob(job_conf);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

#SalesCountryReducer.java

```
package SalesCountry;
import java.io.IOException;
import java.util.*;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.*;

public class SalesCountryReducer extends MapReduceBase implements Reducer<Text, IntWritable,
Text, IntWritable> {

    public void reduce(Text t_key, Iterator<IntWritable> values, OutputCollector<Text,IntWritable>
output, Reporter reporter) throws IOException {
        Text key = t_key;
        int frequencyForCountry = 0;
        while (values.hasNext()) {
            // replace type of value with the actual type of our value
            IntWritable value = (IntWritable) values.next();
            frequencyForCountry += value.get();
        }
        output.collect(key, new IntWritable(frequencyForCountry));
    }
}
```

#SalseMapper.java

```
package SalesCountry;
import java.io.IOException;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.*;

public class SalesMapper extends MapReduceBase implements Mapper <LongWritable, Text, Text,
IntWritable> {
    private final static IntWritable one = new IntWritable(1);

    public void map(LongWritable key, Text value, OutputCollector <Text, IntWritable> output,
Reporter reporter) throws IOException {
        String valueString = value.toString();
        String[] SingleCountryData = valueString.split(",");
        output.collect(new Text(SingleCountryData[7]), one);
    }
}
```

ASSIGNMENT 6: PROGRAM CODE

HIVE Installation

```
stop-all.sh
```

```
hadoop namenode -format
```

```
start-all.sh
```

```
export HADOOP_CLASSPATH=$(hadoop classpath)
```

```
echo $HADOOP_CLASSPATH
```

```
tar xzf apache-hive-3.1.2-bin.tar.gz
```

```
sudo nano .bashrc
```

```
# put following 2 command at end of above file: 1st time installation only
```

```
# export HIVE_HOME=/home/mangal/apache-hive-3.1.2-bin
```

```
# export PATH=$PATH:$HIVE_HOME/bin
```

```
source ~/.bashrc
```

```
sudo nano $HIVE_HOME/bin/hive-config.sh
```

```
# put following command at end of above file: at each new installation
```

```
# export HADOOP_HOME=/home/mangal/hadoop-3.2.3
```

```
hadoop fs -mkdir /tmp
```

```
hadoop fs -chmod g+w /tmp
```

```
hadoop fs -mkdir -p /usr/hive/warehouse
```

```
hadoop fs -chmod g+w /usr/hive/warehouse
```

```
schematool -initSchema -dbType derby
```

```
rm $HIVE_HOME/lib/guava-19.0.jar
```

```
cp $HADOOP_HOME/share/hadoop/hdfs/lib/guava-27.0-jre.jar $HIVE_HOME/lib/
```

```
schematool -initSchema -dbType derby
```

```
hive --hiveconf hive.root.logger=WARN,console
```

```
# or simply give command: hive
```

ASSIGNMENT 7: PROGRAM CODE

Hive Queries

stop-all.sh

hadoop namenode -format

start-all.sh

hive

show databases;

create database dbl;

use dbl ;

create table flight (fno int , year int , dest varchar(10) , delay float);

alter table flight rename to air_flight;

alter table air_flight add columns (new_col1 varchar(10) , new_col2 varchar(10));

#can modify only one column at a time

alter table air_flight change new_col1 new_source1 varchar(15);

#drop columns; list the columns which you want to keep, no direct drop column command

alter table air_flight replace columns (fno int, year int , dest varchar(20), delay float) ;

drop table air_flight;

create database mydb;

use mydb;

create table flight (fno int , year int , dest varchar(10) , delay float)

row format delimited

fields terminated by ','

lines terminated by '\n'

stored as textfile;

desc flight;

insert into flight values (110 , 2008 , "Banglore" , 30.0);

insert into flight values (111 , 2008 , "Pune" , 40.0);

insert into flight values (112 , 2008 , "Mumbai" , 50.0);

insert into flight values (113 , 2008 , "Banglore" , 60.0);

insert into flight values (114 , 2009 , "Pune" , 70.0);

insert into flight values (115 , 2009 , "Mumbai" , 80.0);

select * from flight ;

to load data from a text or csv file into hive table :

#Load Data Statement

load data local inpath "/home/mangal/flight_data.txt" overwrite into table flight;

stop-all.sh

sudo rm -R /tmp/*

sudo rm -r /app/hadoop/tmp # this doesnt run in my case

hdfs namenode -format

start-all.sh

hadoop fs -mkdir /my_dir_in_hadoop

hadoop fs -put /home/mangal/flight_data.txt /my_dir_in_hadoop

hadoop fs -ls /

load data inpath "/my_dir_in_hadoop/flight_data.txt" into table flight;

select * from flight;

creating new table

create table nflight (fno , year int ,
new_source varchar (10))

row format delimited

fields terminated by ','

lines terminated by '\n'

stored as textfile;

insert into nflight values (110 , 2021 , 'Pune');

insert into nflight values (111 , 2022 , 'Pune');

insert into nflight values (112 , 2023 , 'Pune');

joining table

select a.fno , a.year , a.dest , a.delay , b.source
from flight a join nflight b
on (a.fno = b.fno);

sample query find the average departure delay per day in year 2008

select avg(delay) from flight where year = 2008;

exit ; #to leave hive

ASSIGNMENT 8: PROGRAM CODE

Insert here Your Tableau dashbord screenshot

ASSIGNMENT 9 : Program Code

```
# Web Scrapping
import requests
import bs4

# bs4 is used for pulling data out of html and XML in the form of DOM tree
requests1 = requests.get("https://flipkart.com/poco-c31-royal-blue-64-gb-/p/itm19effae969b86")
print("REQUEST 1 = \n", requests1 , "\n")
print("CONTENTS OF REQUEST 1 = \n", requests1.content , "\n")

soup = bs4.BeautifulSoup(request1.text)
<div data-hook="review-collapsed" aria-expanded="false" class="a-expander-content reviewText
review-text-content a-expander-partial-collapse-content">

#reviews = soup.find_all( 'div' , { 'class' : 'a-expander-content reviewText review-text-content a-
expander-partial-collapse-content'}); # or findAll

for review in reviews :
    print(review.get_text() , "\n\n")

<span class="a-size-base a-color-secondary">
<div data-hook="total-review-count" class="a-row a-spacing-medium
averageStarRatingNumerical"><span class="a-size-base a-color-secondary"><!--TODO: Replace
this string with arp-x-ratings 5/22/19 (ShopperExp-5143)-->
    4,323 global ratings</span></div>

ratings = soup.find ( 'div' , { 'class' : 'a-row a-spacing-medium
averageStarRatingNumerical' }).get_text()

print("RATINGS = \n", ratings , "\n")

<i data-hook="review-star-rating" class="a-icon a-icon-star a-star-4 review-rating"><span class="a-
icon-alt">4.0 out of 5 stars</span></i>

# this is i tag and we are trying to extract class tag

individual_ratings = soup.FindAll( 'div' , { 'class' : 'a-icon a-icon-star a-star-4 review-rating' });

for ind_rating in individual_ratings :
    print( indi_rating.get_text() + " \n" )
```

```
# fetching tags
```

```
tags = soup.find ( 'span' , { 'class' : '.....' }).get_text() ;
```

```
print("TAGS = \n", tags , "\n" )
```

```
<div class="a-profile-content"><span class="a-profile-name">Srihari</span></div>
```

for customer names we have got this parent element i.e. div with the 'a-profile-content' as class selector

```
customer_name = soup.findAll( 'p' , { 'class' : '.....' });
```

```
for cust_name in customer_name ;
```

```
    print(cust_name.get_text() + "\n")
```

ASSIGNMENT 10: PROGRAM CODE

Insert here Your Project's Program Code